

Explainer: Quantum Operators and Gottesman Types

Papers: - Gottesman (1998) — *The Heisenberg Representation of Quantum Computers* — arXiv:quant-ph/9807006 - Rand, Sundaram, Singhal & Lackey (2021) — *Gottesman Types for Quantum Programs* — arXiv:2109.02197 (QPL 2020, EPTCS 340) - Sundaram, Rand, Singhal & Lackey (2025) — *Hoare meets Heisenberg: A Lightweight Logic for Quantum Programs* — arXiv:2101.08939

Why This Matters for a Strongly Typed Quantum Language

Designing a strongly typed quantum programming language means finding *static* properties we can check about quantum programs without simulating them — because simulation is exponentially expensive. Gottesman's Heisenberg representation supplies exactly such a property: it tracks how a small generating set of Pauli operators transforms under a circuit, which is enough to pin down structural facts (basis, separability, entanglement, stabilizer membership) in polynomial time for the Clifford fragment. The Rand–Sundaram–Singhal–Lackey insight — that these operator-transformation rules *are* typing judgments — turns this into a genuine type system with subtyping, intersection types, and a top type. That makes it a concrete, implementable foundation for a quantum type checker: judgments are cheap to derive, compose by sequencing, and degrade gracefully (doubling per non-Clifford gate) when the program leaves the Clifford set. The Hoare-logic variant shows the same core extends to a full program logic with pre/postconditions and measurement, which is the shape a verification-oriented language front-end ultimately wants.

States and Operators

Two objects underpin everything that follows: **states** (what the system is) and **operators** (what you do to, or ask about, the system).

Bits become qubits

A classical **bit** is either **0** or **1**. A quantum bit, or **qubit**, can be in a *superposition* — a weighted blend of both at once, written

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle,$$

where α, β are complex amplitudes with $|\alpha|^2 + |\beta|^2 = 1$. The notation $|\cdot\rangle$ ("ket") denotes a state vector; $|0\rangle$ and $|1\rangle$ are the two classical values, now basis vectors. *Measuring* a qubit yields **0** with probability $|\alpha|^2$ or **1** with probability $|\beta|^2$, collapsing the superposition to the observed outcome.

Why states are expensive

One qubit needs two amplitudes; two qubits need four ($|00\rangle, |01\rangle, |10\rangle, |11\rangle$); n qubits need 2^n . This exponential growth is the central obstacle: a 300-qubit state has more amplitudes than there are atoms in the universe, so storing and manipulating the state directly is infeasible. Hence the motivation to avoid tracking states at all.

Operators: gates and observables

An **operator** is a linear map on states, written as a matrix. Two roles concern us:

- **Gates** are *unitary* operators — reversible operations that evolve a state, the quantum analogue of logic gates (AND, NOT, ...). Applying gate U to state $|\psi\rangle$ produces a new state $U|\psi\rangle$. A quantum *program* is just a sequence of gates.
- **Observables / Paulis** are operators we use to *describe* or *question* a state rather than evolve it. The most important are the single-qubit **Pauli operators** X, Y, Z (defined below). They are the vocabulary in which we will state properties.

The key concept relating the two is the **eigenstate**. A state $|\psi\rangle$ is an eigenstate of operator A with eigenvalue λ if

$$A|\psi\rangle = \lambda |\psi\rangle,$$

i.e. A leaves $|\psi\rangle$ unchanged up to a scalar. For Pauli operators the eigenvalues are ± 1 , and " $|\psi\rangle$ is a $+1$ -eigenstate of Z " is a precise, checkable property (the qubit is exactly $|0\rangle$). Tracking such properties is the central activity of this document.

Two pictures of "running a program"

There are two equivalent ways to describe what a gate does:

- **Schrödinger picture** — keep the operators fixed and push the *state* forward: $|\psi\rangle \mapsto U|\psi\rangle$. Intuitive, but it forces you to carry the exponentially large state.
- **Heisenberg picture** — keep the *state* fixed and track how *operators* change: $A \mapsto UAU^\dagger$. Less familiar, but it requires following only a handful of operators, which is dramatically cheaper.

Both give identical measurement predictions; they differ only in bookkeeping. Everything below lives in the Heisenberg picture, with the type system built on top of it.

Key Ideas

1. Track operators, not states (the Heisenberg picture)

A state on n qubits needs $2^n - 1$ complex amplitudes — the exponential wall. The Heisenberg representation sidesteps this by evolving *operators*. Applying a unitary U to the state, for any observable N ,

$$UN|\psi\rangle = (UNU^\dagger)U|\psi\rangle,$$

so a gate U acts on operators by conjugation:

$$N \longrightarrow UNU^\dagger.$$

This map is **linear** and **multiplicative**:

$$U(MN)U^\dagger = (UMU^\dagger)(UNU^\dagger).$$

Therefore it is fully determined by its action on a *generating set* of operators — no need to track all 4^n Paulis, just the generators.

2. The Pauli group as the bookkeeping basis

Gottesman uses the Pauli group, built from

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

The algebra is trivial to mechanise:

- $X^2 = Y^2 = Z^2 = I$
- $Y = iXZ$ — so you only ever track X and Z ; Y is derived
- they anticommute: $ZX = -XZ$

To specify *any* n -qubit operation it suffices to follow the $2n$ generators $\{X_1, \dots, X_n, Z_1, \dots, Z_n\}$. Linear bookkeeping replaces exponential state.

3. The Clifford group: where it stays cheap

Operator-tracking is cheap only while conjugation maps Paulis *back to* Paulis. The gates with this property form the **Clifford group**, generated by three gates:

Gate	Action on Paulis
H (Hadamard)	$X \rightarrow Z, \quad Z \rightarrow X$
S (phase)	$X \rightarrow Y, \quad Z \rightarrow Z$
CNOT	$X \otimes I \rightarrow X \otimes X, \quad I \otimes X \rightarrow I \otimes X, \quad Z \otimes I \rightarrow Z \otimes I, \quad I \otimes Z \rightarrow Z \otimes Z$

Following the $2n$ generators through a Clifford circuit costs only polynomial work — this is the **Gottesman–Knill theorem**: Clifford circuits are efficiently classically simulable.

Worked example (SWAP from three CNOTs). Tracking the generators through $\text{CNOT}(1 \rightarrow 2); \text{CNOT}(2 \rightarrow 1); \text{CNOT}(1 \rightarrow 2)$ gives $X_1 \rightarrow X_2, X_2 \rightarrow X_1, Z_1 \rightarrow Z_2, Z_2 \rightarrow Z_1$ — i.e. the circuit swaps qubits 1 and 2, deduced without ever writing a state vector.

4. The leap: arrows are typing judgments

Rand–Sundaram–Singhal–Lackey observed that a rule like $H : Z \rightarrow X$ *reads like a type signature*. So treat Pauli operators as **types** and gates as **functions between types**, with the Heisenberg arrow as the typing arrow.

Interpretation on basis states (their Proposition 1): if $U : A \rightarrow B$, then U takes every eigenstate of A to an eigenstate of B . Concretely, $H : Z \rightarrow X$ means " H takes a qubit in the Z basis ($|0\rangle, |1\rangle$) to a qubit in the X basis ($|+\rangle, |-\rangle$)."
A type is a *promise about the eigenstate structure of the output*, not a full description of the state.

Technical Details

Deriving composite gates

With types for H, S, CNOT you derive types for everything else by composition. Using $;$ for sequencing and $\$$ for forward composition ($f \$ g \equiv g \circ f$):

- $Z = S; S \Rightarrow Z : X \rightarrow -X, Z : Z \rightarrow Z$
- $X = H; Z; H \Rightarrow X : X \rightarrow X, X : Z \rightarrow -Z$
- $Y = S; X; Z; S \Rightarrow Y : X \rightarrow -X, Y : Z \rightarrow -Z$
- CZ, SWAP , etc. follow mechanically.

Intersection types

A single gate has several arrows at once; intersection types ($\cap$) record the most descriptive type. For CNOT:

$$\text{CNOT} : (X \otimes I \rightarrow X \otimes X) \cap (I \otimes X \rightarrow I \otimes X) \cap (Z \otimes I \rightarrow Z \otimes I) \cap (I \otimes Z \rightarrow Z \otimes Z).$$

The standard subtyping rules for $\rightarrow$ and $\cap$ (e.g. $\cap\text{-ARR-DIST}$) let you weaken or combine these as needed. This connects directly to the **stabilizer formalism** of error correction.

Separability and entanglement tracking

A type $U_k := I^{\otimes(k-1)} \otimes U \otimes I^{\otimes(n-k)}$ certifies that qubit k is an eigenstate of the single-qubit Pauli U and is **unentangled** from the rest (their Corollary 1). They introduce $\times$ for this separable structure, e.g.

$Z \times Z \times Z \times Z =$ four independent classical bits.

GHZ example. Typing the circuit that builds $|000\rangle + |111\rangle$ from $|000\rangle$ gives $Z_1 \rightarrow X \otimes X \otimes X$ — entanglement created. Applying later CNOTs walks the type back down to the fully separable $Z \times Z \times X$ — entanglement *destroyed*. The type system tracks both directions for free, where density-matrix approaches (Honda) scale badly and basis-only abstractions (Perdrix) collapse to $\top$ after a few gates.

Superdense coding type-checks to $Z \times Z \times Z \times Z \rightarrow Z \times Z \times Z \times Z$: four classical bits in, four out.

Beyond Clifford: the $\top$ type

Clifford circuits are not universal. To type the T ($\pi/8$) gate and Toffoli, add a top type $\top$:

$$T : Z \rightarrow Z, \quad T : X \rightarrow \top.$$

Crucially $\top$ is an **annihilator** (unlike I , which is an identity): a non-Clifford gate can smear a sharp Pauli type into "anything." Cost model: analysis is **linear** in the number of Clifford gates and **doubles per non-Clifford gate** in the worst case — the precise boundary of efficient operator-tracking.

The Hoare-logic reframing

Hoare meets Heisenberg recasts $A \rightarrow B$ as a Hoare triple

$$\{A\} U \{B\},$$

read " U maps a $+1$ -eigenstate of A to a $+1$ -eigenstate of B ." A predicate $P(|\psi\rangle)$ means $|\psi\rangle$ lies in the image of the projector $\Pi_P^+ = \frac{1}{2}(I + P)$ — i.e. $|\psi\rangle$ is stabilized by P . The logic adds:

- $\cap$ (intersection / conjunction): simultaneous $+1$ -eigenstates — requires the Paulis to **commute** (non-commuting Paulis anticommute and share no eigenvector);
- $\uplus$ (disjoint union): the two branches of a probabilistic measurement outcome.

Applications: certifying ancilla disposal, separability across a bipartition, gate transversality for stabilizer codes, post-measurement states, and even a lower bound on the number of T gates needed for a multiply-controlled Z .

The Y rule in deduction form

Because $Y = iXZ$, the action on Y is forced by the actions on X and Z :

$$\frac{\{X\} U \{A\} \quad \{Z\} U \{B\}}{\{Y\} U \{iAB\}}.$$

This is the recurring shape of the rules: deduce the third Pauli from the other two.

Implications for Language Design

- **Typing judgments as the core abstraction.** A rule `gate : A → B` is both the operational semantics (Heisenberg conjugation) and the type rule. A type checker can derive the postcondition type of a straight-line Clifford program in linear time by threading the $2n$ generators — no state vector, no exponential blowup.
- **Subtyping and intersection types are essential.** Multiple arrows per gate ($\cap$) give the most descriptive type; subtyping (e.g. $\cap$ -ARR-DIST) lets the checker weaken to exactly the property a programmer cares about (e.g. separability via $\times$). These are standard PL constructs, so they slot into a conventional type-system implementation.
- **A principled cost model for type checking.** Checking is polynomial on the Clifford fragment and worst-case doubles per non-Clifford (T , Toffoli) gate via the $\top$ annihilator. This gives the language a clear, predictable complexity contract for static analysis and tells you where to demand annotations or magic-state structure.
- **Properties worth exposing as types.** Separability / non-entanglement ($\times$), safe ancilla disposal, stabilizer-code transversality, and post-measurement structure ($\uplus$) are all checkable in this system — a strong candidate feature set for a verification-oriented quantum language.
- **Formalization precedent.** The 2021 type system is mechanized in Coq (`inQWIRE/GottesmanTypes`), and the 2025 logic gives soundness proofs — useful references when proving the metatheory (soundness, principal types) of a new language.